

Problem Set: Dynamic Programming

1 Problem B: Damerau–Levenshtein Distance

The Damerau–Levenshtein Distance is an extension of the classic edit distance problem. In addition to the standard operations, it introduces a fourth operation:

- **Insertion**
- **Deletion**
- **Substitution**
- **Transposition:** Swapping two *adjacent* characters in a single step (e.g., converting "ca" to "ac" costs 1).

Given two strings A and B , compute the minimum number of operations required to transform A into B .

Input

Two strings A and B

Output

An integer representing the Damerau–Levenshtein distance between A and B

Example

Input:

$A = \text{"ca"}$

$B = \text{"ac"}$

Output:

1

Hint (Designing the Recurrence)

Define a DP table $dp[i][j]$ as the minimum cost to convert the first i characters of A into the first j characters of B .

- Start from the standard edit distance recurrence (insertion, deletion, substitution).

- Then consider an additional case for transposition:
 - Check if $i > 1$ and $j > 1$
 - Check whether the last two characters are swapped: [$A[i] = B[j-1]$ and $A[i-1] = B[j]$]
- If this condition holds, relate the answer to a smaller subproblem by removing both characters involved in the swap.
- Carefully compare all possible choices to take the minimum.

Task

- Formulate the recurrence relation.
- Implement a dynamic programming solution.

Additional Test Cases

Test Case 1

A = "abcdef"

B = "abcfde"

Output:

2

Explanation: The transformation involves reordering the substring "def" into "fde". Since only adjacent swaps are allowed, this cannot be done in one step. Instead:

- Swap 'e' and 'f': "abcdef" $\rightarrow$ "abcdfe"
- Swap 'd' and 'f': "abcdfe" $\rightarrow$ "abcfde"

Thus, two transpositions are required, giving a total cost of 2.

Test Case 2

A = "kitten"

B = "sitting"

Output:

3

Explanation: The optimal sequence is:

- Substitute 'k' with 's'
- Substitute 'e' with 'i'
- Insert 'g' at the end

No transposition helps in this case, so the result is the same as standard edit distance.

Test Case 3

A = "abcd"

B = "abdc"

Output:

1

Explanation: The last two characters 'c' and 'd' are adjacent and swapped. A single transposition converts "abcd" to "abdc" in one step. Although two substitutions could also achieve this, transposition is cheaper, so the optimal cost is 1.